

스트림릿 설정 환경

파이썬 몇 줄로 웹앱을 만드는 Streamlit.

도커(docker-compose)로 개발 환경을 통째로 준비하는 법부터.

Q1 스트림릿이 뭔데?

Streamlit(스트림릿)은 파이썬 코드만으로 웹앱을 만드는 도구입니다. HTML·CSS·자바스크립트를 몰라도, `st.title(...)`, `st.slider(...)` 처럼 한 줄씩 쓰면 그대로 화면 요소가 됩니다.

대신 정적인 HTML과 달리 서버가 돌아가야 합니다. 그래서 `streamlit run app.py` 로 앱을 켜고, 브라우저로 접속해서 봅니다.



파이썬 코드 → 스트림릿 서버 → 브라우저 웹앱.

스트림릿 = "파이썬만 알면 나도 웹앱을 만든다"

데이터 분석 결과를 남에게 보여줄 화면을 가장 빠르게 만드는 도구.

Q2 왜 도커로 개발하나

"내 컴퓨터에선 되는데요?" 문제를 없애기 위해서예요. **도커(Docker)**는 파이썬 버전·라이브러리·실행 명령을 **상자(컨테이너)** 하나에 **통째로** 담습니다. 그 상자만 있으면 누구 컴퓨터에서든 **똑같이** 돌아가죠.

프로젝트 폴더 구조 (01_INIT)

```
01_INIT/  
├─ app.py           # 스트림릿 메인 앱  
├─ requirements.txt # 설치할 라이브러리 목록(버전 고정)  
├─ docker-compose.yml # 도커 실행 설정  
├─ dataSet/  
│   └─ 학생점수.csv # 데이터 파일 폴더  
└─ pages/  
    ├─ page1.py  
    ├─ page2.py  
    └─ page3.py
```

`docker-compose.yml` 한 파일이 "이 상자를 어떻게 만들고 실행할지"를 적어둔 설계도입니다. 다음 장에서 뜯어봅시다.

Q3 docker-compose.yml 뜯어보기

실제 수업에서 쓰는 설정입니다. 한 줄씩 무슨 뜻인지 주석으로 달았어요.

```
services:
  streamlit:
    image: python:3.10-slim          # 파이썬 3.10 가벼운 이미지 사용
    working_dir: /app               # 컨테이너 안 작업 폴더
    ports:
      - "8601:8501"                # 브라우저 8601 → 컨테이너 8501
    volumes:
      - ./:/app                    # 내 폴더를 컨테이너에 연결(저장 시 즉시 반영)
      - ./requirements.txt:/tmp/requirements.txt
    environment:
      - STREAMLIT_SERVER_FILE_WATCHER_TYPE=poll  # 도커에서 변경 감지 안정화
      - STREAMLIT_SERVER_RUN_ON_SAVE=true        # 저장하면 자동 새로고침
    command: >
      bash -lc "pip install --no-cache-dir -r /tmp/requirements.txt &&
        streamlit run app.py
        --server.port=8501 --server.address=0.0.0.0"
```

포트 **8601:8501** 가 헷갈리죠? 앞(8601)이 내 브라우저 주소, 뒤(8501)가 컨테이너 속 스트림릿입니다. 그래서 접속은 **http://localhost:8601**로 해요. (스트림릿 기본 포트가 8501이라 안쪽은 8501 고정)

이 설정의 3가지 핵심

- ① **volumes** `./:/app` — 내 코드 폴더를 상자에 연결. 코드를 고치면 상자 안에도 바로 반영
- ② **RUN_ON_SAVE=true** — 저장만 하면 브라우저가 스스로 새로고침
- ③ **command** — 상자가 켜질 때 라이브러리 설치 후 `streamlit run app.py` 실행

Q4 requirements.txt — 버전 고정

설치할 라이브러리 목록입니다. 버전을 꼭 집어 고정하는 게 핵심이에요. 안 그러면 나중에 최신 버전이 깔려 "어제는 됐는데 오늘 안 됨"이 생깁니다.

```
matplotlib
streamlit==1.28.0
pandas==2.1.0
numpy==1.24.3
scikit-learn==1.3.0
joblib==1.3.2
lightgbm==4.1.0
watchdog
```

라이브러리	역할
streamlit	웹앱 화면을 만드는 주인공
pandas · numpy	표·숫자 데이터 다루기
scikit-learn · lightgbm	머신러닝 모델(학습·예측)
joblib	학습한 모델을 파일(.pkl)로 저장/불러오기 — 03장에서 사용
watchdog	파일 변경 감지(저장 시 새로고침 도움)

버전은 팀 전체가 같아야 합니다. 특히 `scikit-learn · lightgbm` 은 모델(.pkl)을 만든 버전과 앱에서 불러오는 버전이 다르면 오류가 나요. (03장에서 다시 강조)

Q5 실행하기 — 딱 한 줄

설정 파일이 준비됐으면 실행은 간단합니다. 프로젝트 폴더에서:

```
# 상자를 만들고 앱을 실행 (처음엔 라이브러리 설치로 조금 걸림)
docker compose up

# 끄기: Ctrl+C 후
docker compose down
```

그다음 브라우저에서 → `http://localhost:8601`

코드를 고치고 저장하면(`RUN_ON_SAVE`) 화면이 스스로 새로고침됩니다. 앱을 껐다 켤 필요가 없어요.



```
$ docker compose up
```

```
You can now view your Streamlit app → http://localhost:8601
```

Q6 여러 페이지 만들기 — pages 폴더

앱이 커지면 화면을 나누고 싶죠. 스트림릿은 `pages/` 폴더에 파이썬 파일을 넣기만 하면 사이드바에 메뉴가 자동으로 생깁니다.

```
# pages/page1.py - 파일 하나가 메뉴 하나
import streamlit as st

st.title("첫번째 페이지 입니다")
```

사이드바

app (메인)

page1

page2

page3

파일 이름이
그대로 메뉴가 됩니다

Q7 한 장 정리

파 일	하 는 일
<code>app.py</code>	스트림릿 메인 화면 코드
<code>docker-compose.yml</code>	실행 환경 설계도(이미지·포트·볼륨·명령)
<code>requirements.txt</code>	설치 라이브러리 버전 고정 목록
<code>pages/*.py</code>	추가 페이지 → 사이드바 자동 메뉴

이것만 기억하면 됩니다

- ① 스트림릿은 파이썬으로 만드는 웹앱, 서버를 켜서 본다
- ② 도커로 환경을 통째로 담아 "내 컴퓨터에선 되는데" 문제를 없앤다
- ③ 실행은 `docker compose up` → `localhost:8601`
- ④ `requirements.txt` 는 버전 고정, `pages/` 는 자동 메뉴

Q8 단계별로 구축하기

아래 순서 그대로 따라 하면 스트림릿 개발 환경이 완성됩니다. 각 단계에서 만드는 파일·명령을 복사 버튼으로 그대로 만들고, 코드 해석 ▼으로 한 줄씩 뜻을 확인하세요.

STEP 1 프로젝트 폴더와 하위 폴더 만들기

```
# 프로젝트 폴더를 만들고 그 안으로 이동
mkdir streamlit_app
cd streamlit_app

# 하위 폴더 만들기 (데이터 / 추가 페이지)
mkdir dataSet pages

# 이제 이 안에 만들 파일들:
# app.py · requirements.txt · docker-compose.yml
```

```
mkdir streamlit_app / cd streamlit_app
```

작업할 프로젝트 폴더를 만들고 그 안으로 들어갑니다. 앞으로 모든 파일은 여기에 만들어요.

```
mkdir dataSet pages
```

데이터 파일을 둘 **dataSet**과, 추가 화면을 둘 **pages** 폴더를 미리 만듭니다.

```
# app.py · requirements.txt · docker-compose.yml
```

다음 단계부터 이 세 파일을 하나씩 만들 거예요. 순서대로 채우면 됩니다.

STEP 2 requirements.txt — 설치할 라이브러리(버전 고정)

```
# requirements.txt
matplotlib
streamlit==1.28.0
pandas==2.1.0
numpy==1.24.3
scikit-learn==1.3.0
joblib==1.3.2
lightgbm==4.1.0
watchdog
```

```
streamlit==1.28.0 ...
```

각 라이브러리 뒤에 **==버전**으로 정확히 고정합니다. 팀 전체가 같은 버전을 써야 "나만 안 됨"이 없어요.

```
scikit-learn / lightgbm / joblib
```

03장에서 모델(.pkl)을 만들고 불러올 때 씁니다. **모델 만든 버전 = 앱 버전**이어야 안전!

```
watchdog
```

파일 변경 감지를 도와, 저장 시 자동 새로고침이 잘 되게 해줍니다.

STEP 3 app.py — 스트림릿 첫 화면

```
# app.py
import streamlit as st

st.title("첫 스트림릿 앱")
st.write("파이썬 몇 줄로 웹앱을 만듭니다.")
st.metric("오늘 기온", "25°C", "+2°C")
```

```
import streamlit as st
```

스트림릿을 불러옵니다. 보통 **st**로 줄여 써요. 앞으로 모든 화면 요소는 **st.무엇()** 형태.

```
st.title(...) / st.write(...)
```

큰 제목과 일반 글을 화면에 찍습니다. 코드 순서 = 화면에 나타나는 순서예요.

```
st.metric("오늘 기온", "25°C", "+2°C")
```

숫자 카드를 예쁘게 보여줍니다. 세 번째 값(+2°C)은 변화량이라 초록/빨강으로 표시돼요.

STEP 4 docker-compose.yml — 실행 환경 설계도

```
# docker-compose.yml
services:
  streamlit:
    image: python:3.10-slim
    working_dir: /app
    ports:
      - "8601:8501"          # 브라우저 8601 → 컨테이너 8501
    volumes:
      - ./:/app              # 내 폴더 연결(저장 시 즉시 반영)
      - ./requirements.txt:/tmp/requirements.txt
    environment:
      - STREAMLIT_SERVER_FILE_WATCHER_TYPE=poll
      - STREAMLIT_SERVER_RUN_ON_SAVE=true
    command: >
      bash -lc "pip install --no-cache-dir -r /tmp/requirements.txt &&
        streamlit run app.py
        --server.port=8501 --server.address=0.0.0.0"
```

`image: python:3.10-slim`

상자의 바탕이 될 파이썬 3.10 이미지. **slim**은 용량을 줄인 가벼운 버전이에요.

`ports: "8601:8501"`

브라우저 주소(8601)와 컨테이너 속 스트림릿(8501)을 연결합니다. 접속은 **localhost:8601**.

`volumes: ./:/app`

지금 폴더를 상자 안 `/app` 에 연결. 코드를 고치면 상자에도 바로 반영돼요.

`environment: RUN_ON_SAVE=true`

파일을 저장하면 브라우저가 자동으로 새로고침됩니다. 개발이 편해져요.

`command: pip install ... && streamlit run app.py`

상자가 켜질 때 라이브러리를 설치한 뒤 앱을 실행합니다. `0.0.0.0` 은 바깥에서 접속 허용.

STEP 5 실행하기 — docker compose up

프로젝트 폴더에서 실행 (처음엔 라이브러리 설치로 조금 걸림)

```
docker compose up
```

브라우저에서 접속 → <http://localhost:8601>

코드를 고치고 저장하면 자동 새로고침(RUN_ON_SAVE)

끄기

```
docker compose down
```

```
docker compose up
```

STEP 2~4의 세 파일이 준비되면 이 한 줄로 상자를 만들고 앱을 실행합니다. 처음엔 라이브러리 설치로 시간이 조금 걸려요.

```
http://localhost:8601
```

브라우저로 이 주소에 접속하면 앱이 보입니다. (8601 = docker-compose에서 연 브라우저 포트)

```
docker compose down
```

앱을 끄고 상자를 정리합니다. 다시 켜려면 `up` 을 또 실행하면 돼요.

Q9 전체 코드 한눈에

구축에 필요한 **파일 전체**를 모았습니다. 위에서 아래 순서대로 만들면 됩니다.

requirements.txt

```
matplotlib
streamlit==1.28.0
pandas==2.1.0
numpy==1.24.3
scikit-learn==1.3.0
joblib==1.3.2
lightgbm==4.1.0
watchdog
```

app.py

```
import streamlit as st

st.title("첫 스트림릿 앱")
st.write("파이썬 몇 줄로 웹앱을 만듭니다.")
st.metric("오늘 기온", "25°C", "+2°C")
```

docker-compose.yml

```
services:
  streamlit:
    image: python:3.10-slim
    working_dir: /app
    ports:
      - "8601:8501"
    volumes:
      - ./:/app
      - ./requirements.txt:/tmp/requirements.txt
    environment:
      - STREAMLIT_SERVER_FILE_WATCHER_TYPE=poll
      - STREAMLIT_SERVER_RUN_ON_SAVE=true
    command: >
      bash -lc "pip install --no-cache-dir -r /tmp/requirements.txt &&
        streamlit run app.py
        --server.port=8501 --server.address=0.0.0.0"
```

▶ 실행 (터미널)

```
# 프로젝트 폴더에서 실행
docker compose up

# 브라우저 접속 → http://localhost:8601
```

